From Offline to Online Inference: Why Serving Is Hard—and How vLLM Helps

vLLM Hong Kong Meetup

7 March 2026

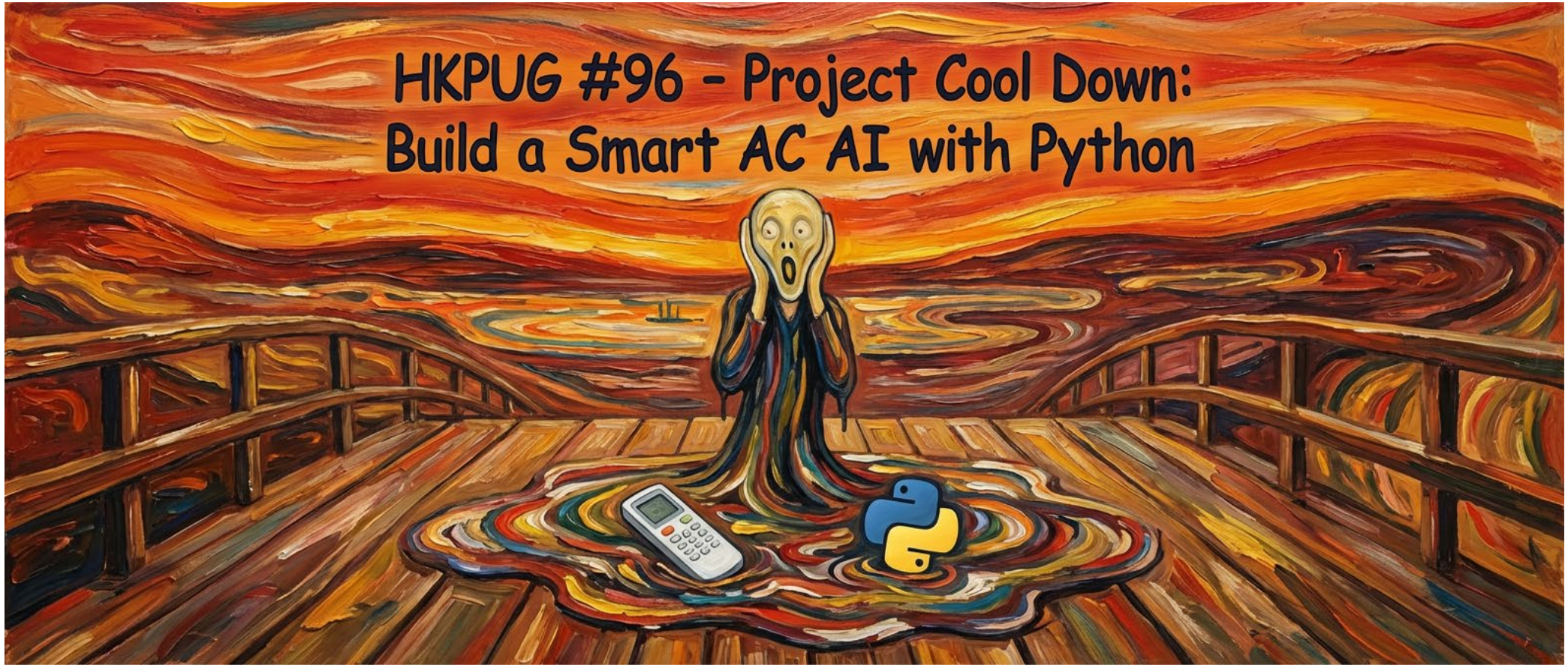
Who am I

- **Meetup Organizer of HK Python User Group**
- Data Scientist at IBM
- Researcher in Astrophysics

Connect if you are interested :D

Hong Kong Python User Group

Scan to reserve your seat!



QR Codes

Slides for Today

Code for Today

What is vLLM

- Fast and easy-to-use library for LLM inference and serving
- Originally developed in the Sky Computing Lab at UC Berkeley
- Evolved into a community-driven project

Inferencing with vLLM [1]

- OpenAI-Compatible Server

Hosting your own LLM with just one line of command
(given you have enough hardware: GPU, VRAM, etc.)

vLLM

Search

GitHub v0.16.0 71.4k 13.8k

Home | User Guide | Developer Guide | Benchmarking | API Reference | CLI Reference | Community

User Guide

General
- vLLM V1
- Frequently Asked Questions
- Production Metrics
- Reproducibility
- Security
- Troubleshooting
- Usage Stats Collection

Inference and Serving

OpenAI-Compatible Server

vLLM provides an HTTP server that implements OpenAI's Completions API, Chat API, and more! This functionality lets you serve models and interact with them using an HTTP client.

In your terminal, you can install vLLM, then start the server with the `vllm serve` command. (You can also use our Docker image.)

```
vllm serve NousResearch/Meta-Llama-3-8B-Instruct \
  --dtype auto \
  --api-key token-abc123
```

Table of contents
- Supported APIs
- Chat Template
- Extra Parameters
- Extra HTTP Headers
- Offline API Documentation
- API Reference
  - Completions API
    - Extra parameters
  - Chat API
    - Extra parameters

Inferencing with vLLM [2]

Full workflow: start server → call endpoint → see a response.

```
(vllm-py312) wyhwong@wyhwong-desktop:~/projects/RedHat-vLLM-inferencing$ uv run --active --with vllm vllm serve Qwen/Qwen3-4B --max_model_len 8192
(APIServer pid=8833) INFO:     Started server process [8833]
(APIServer pid=8833) INFO:     Waiting for application startup.
(APIServer pid=8833) INFO:     Application startup complete.
```

```python
import requests

response = requests.post(
    url="http://localhost:8000/v1/chat/completions",
    json={
        "model": "Qwen/Qwen3-4B",
        "messages": [
            {"role": "user", "content": "Who are you? Reply in one sentence."},
        ],
    },
)

data = response.json()
print(data["choices"][0]["message"]["content"])
```

✓ 1.4s Python

```
I am Qwen, a large-scale language model developed by Alibaba Group's Tongyi Lab, capable of text generation, answering questions, and engaging in conversations.
```

Is vLLM fast? How fast?

How Fast is vLLM [1]

- Ollama as baseline
- apples-to-apples comparison

vLLM

How Fast is vLLM [2]

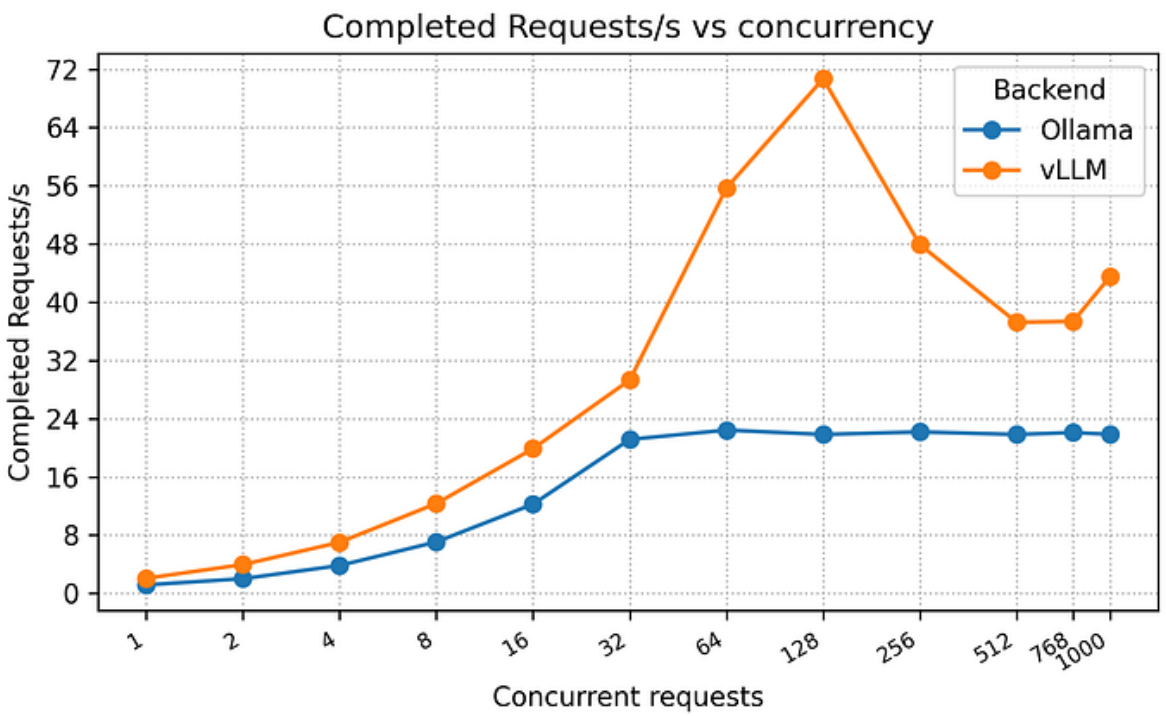
Metric	Unit	Ollama	vLLM	Enhancement
Throughput	Request/second	**0.144**	**0.304**	**+111.1%**
	Token/second	**216.00**	**311.48**	**+44.2%**
Concurrency	/	8	20	/

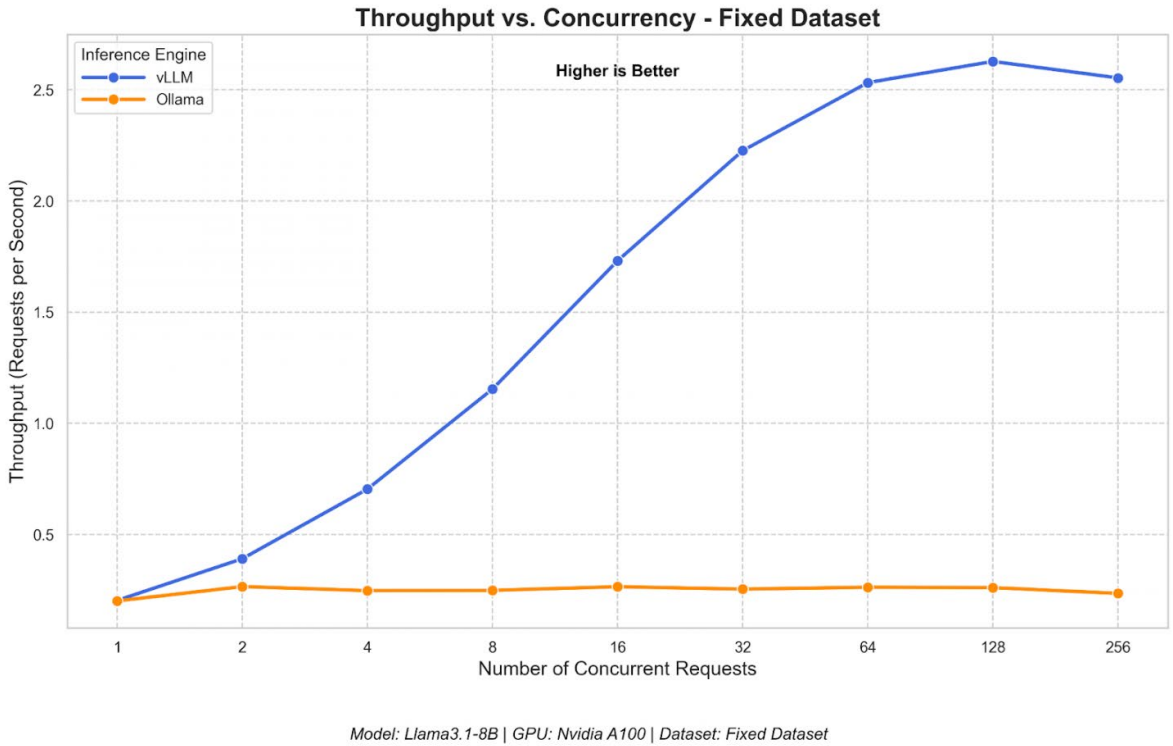
1.5-2x performance!

Setup Summary:

- Model: Qwen/Qwen3-4B (vLLM), qwen3:4b (Ollama)
- GPU: Nvidia RTX 4060 Ti (16GB VRAM)
- Data: 50 requests randomly sampled from Alpaca

vLLM Speed in the Wild





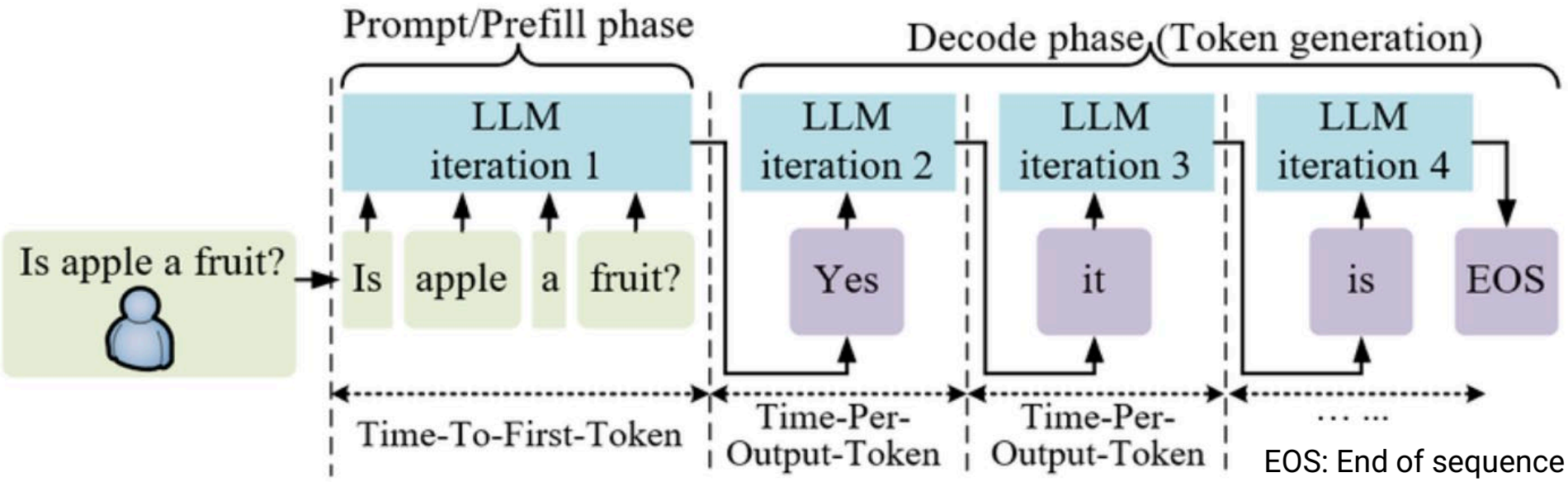
Roughly 1.5x faster at n=20

Roughly 7x faster at n=20

Left: [Performance vs Practicality: A Comparison of vLLM and Ollama]
Right: [Ollama vs. vLLM: A deep dive into performance benchmarking]

LLM Inference Basics

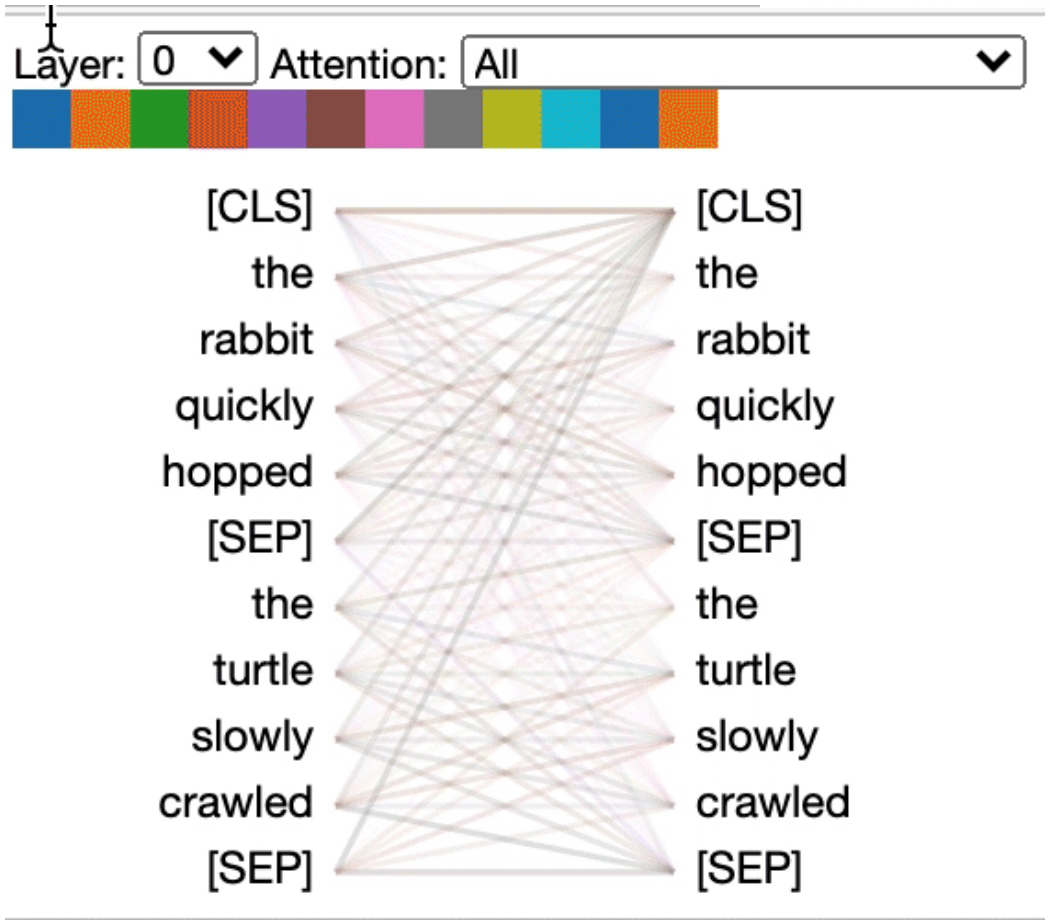
- LLM: An autoregressive model that predicts a probability distribution over the next token, given previous tokens.
- Answering a prompt requires multiple decode steps, **one per generated token**.



[1] Generative AI as a Service in 6G Edge-Cloud: Generation Task Offloading by In-context Learning, 2024.

LLM Inference Basics: Attention

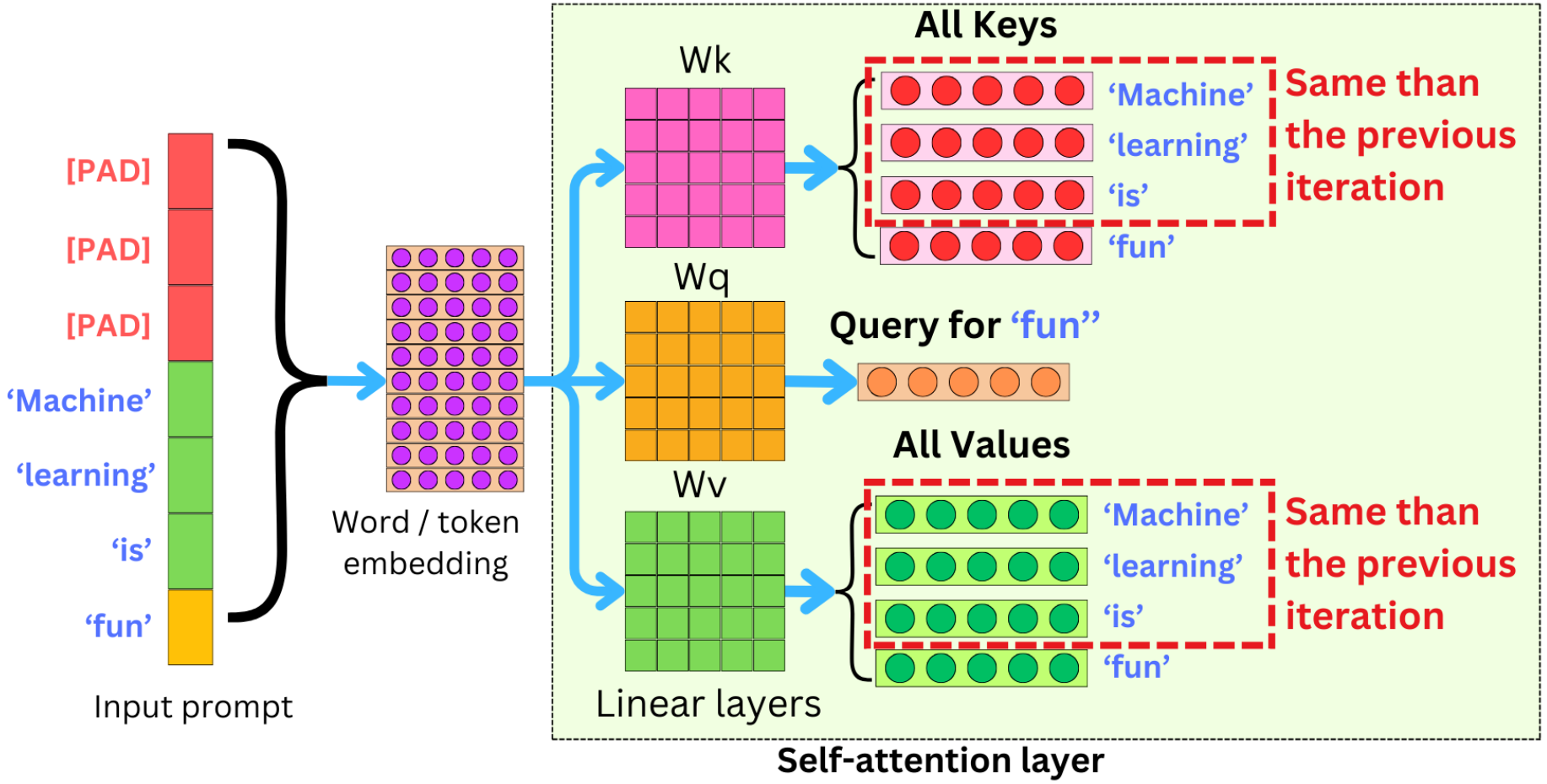
- Attention:
  - key components in every LLMs
  - each token could attend to each token
- Each layer: $O(N^2)$
- Each inference: $O(N^2 L)$
- Each request: $\boldsymbol{O(N^3 L)}^*$



*without any caching, N is length of the sequence.

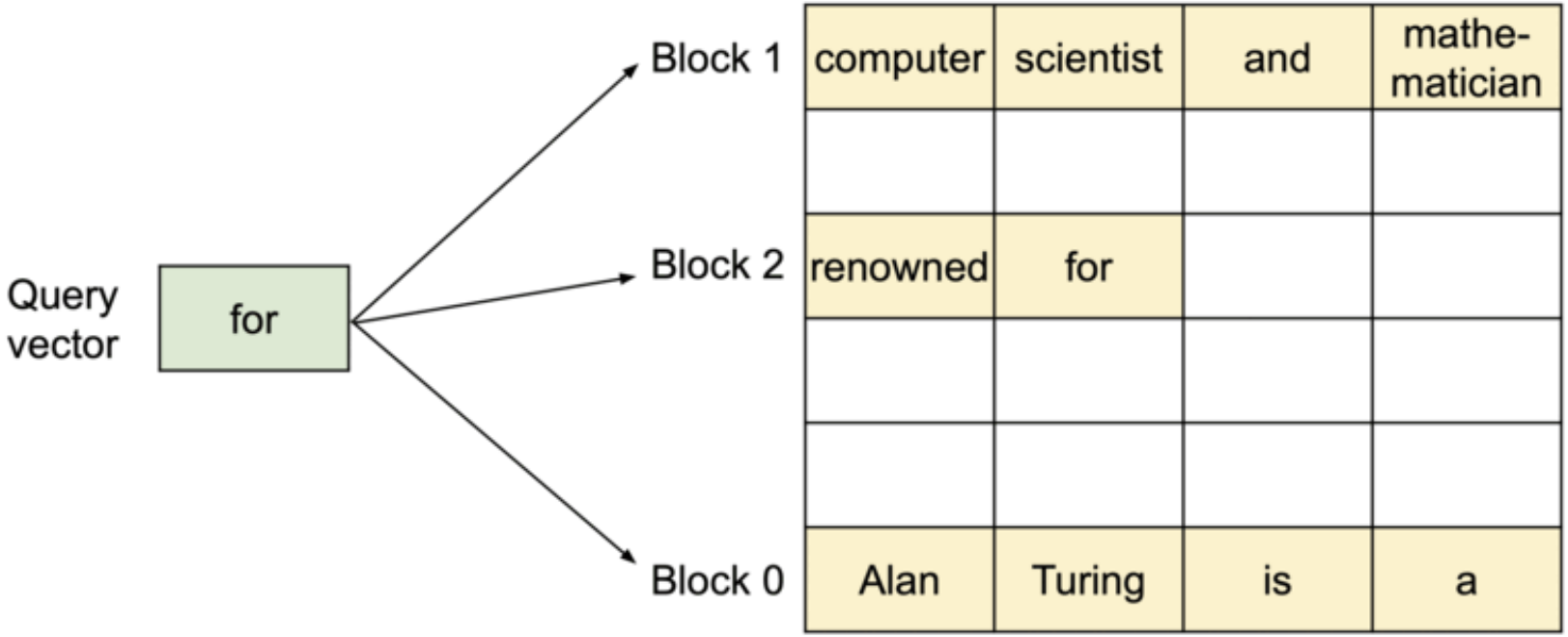
[1] https://github.com/jessevig/bertviz
* This is a gif, please expect it won't show properly in pdf.

LLM Inference Basics: KV Cache



[1] How To Reduce LLM Decoding Time With KV-Caching, 2024.

Efficient caching in vLLM: Paged Attention



[1] vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention, 2023.
* This is a gif, please expect it won't show properly in pdf.

Batching Prompts for LLM Inference

Input 1	I	go	to	school	by
Input 2	My	name	is	Peter	
Input 3	vLLM	is			

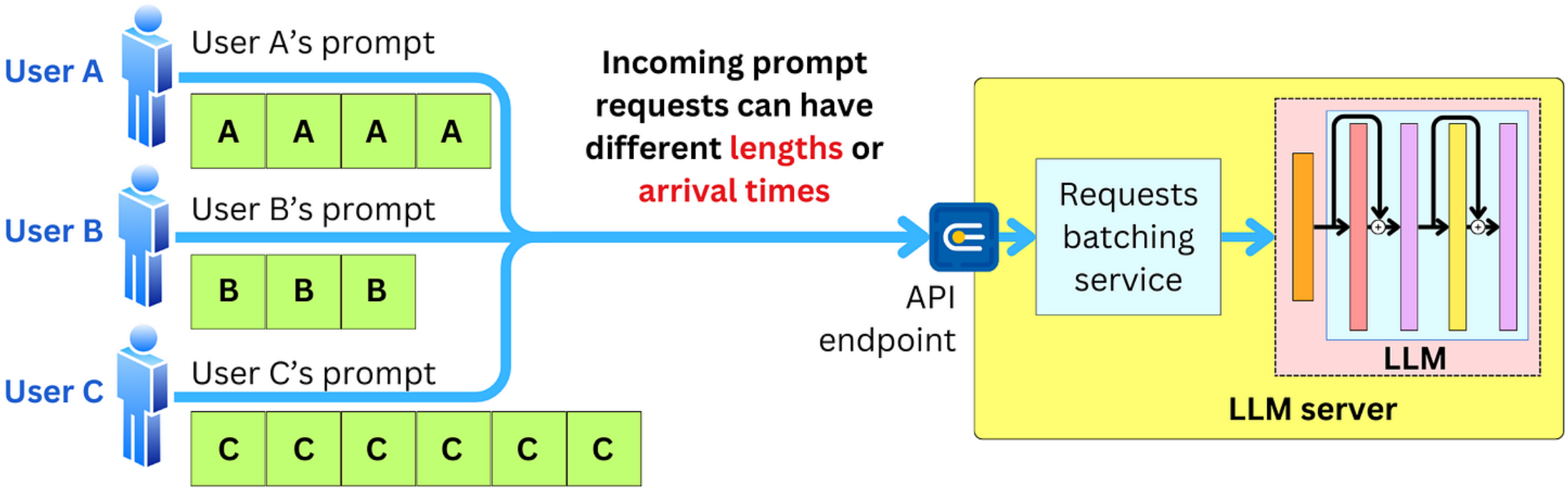
Padding

I	go	to	school	by
<PAD>	My	name	is	Peter
<PAD>	<PAD>	<PAD>	vLLM	is

Batch Inference

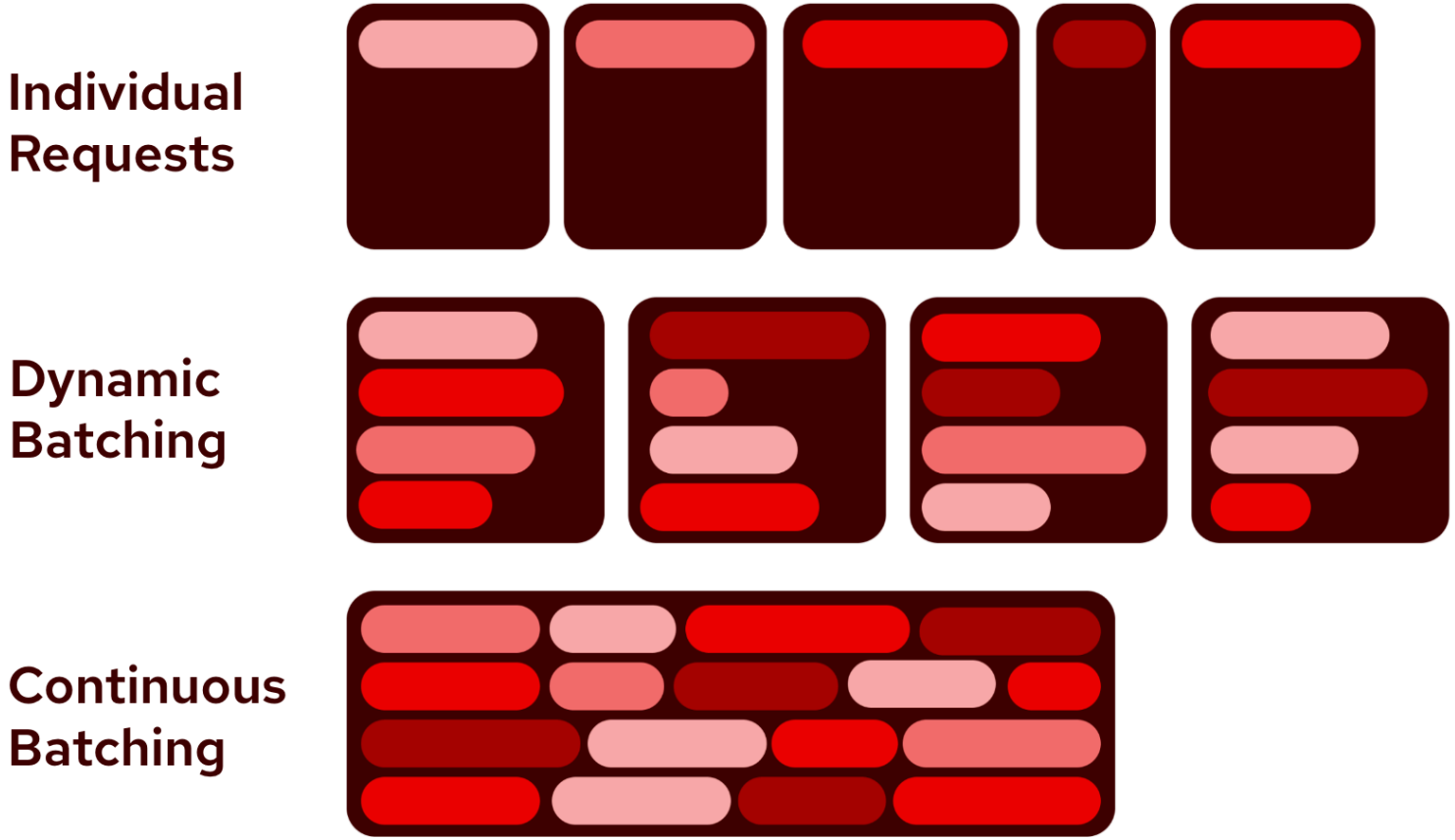
I	go	to	school	by	bus
<PAD>	My	name	is	Peter	Parker
<PAD>	<PAD>	<PAD>	vLLM	is	fast

Concurrency in Online Inference



[1] How to Scale LLM Applications With Continuous Batching, 2024.

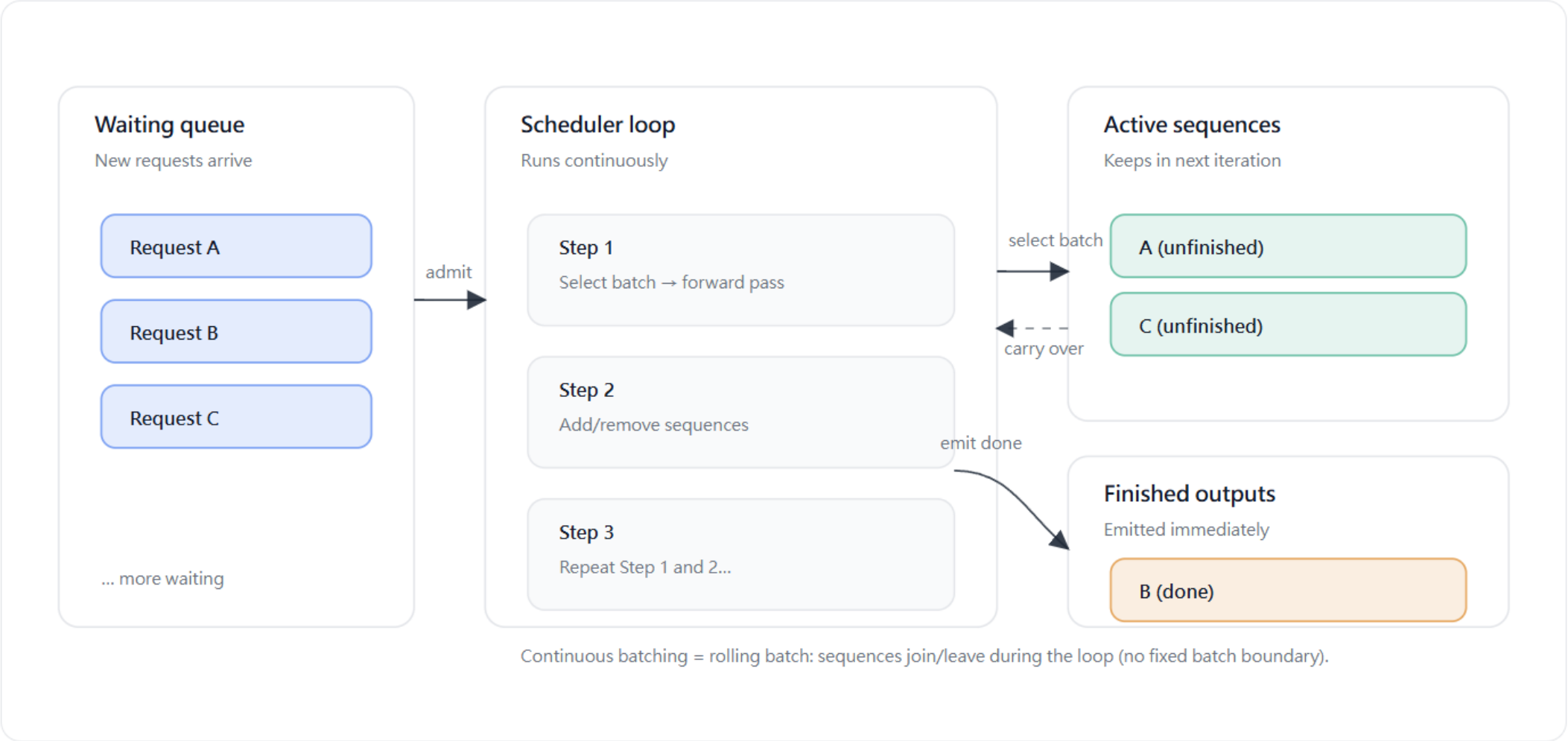
vLLM: Continuous Batching



Looks simple
But how?

[1] Meet vLLM: For faster, more efficient LLM inference and serving, 2025.

Elegant Engineering Solution in vLLM



Waiting · Active (unfinished) · Finished · Solid = main flow, dashed = carry-over

Further Acceleration: Beyond Basics

- Ultimately, we want inference to be fast.
- What techniques can push speed further than:
  - Prompt engineering
  - Quantization
  - Speculative decoding
  - Distilled model (and more)

Prompt Engineering

- Optimize prompt to guide model to generate desired outputs, e.g. more concise, more creative, etc.

- **Easiest trick** to largen throughput (request/second).

```python
response = requests.post(
    url="http://localhost:8000/v1/chat/completions",
    json={
        "model": "Qwen/Qwen3-4B",
        "messages": [
            {"role": "user", "content": "Who are you? Reply in one sentence."},
        ],
    },
)
```

```python
response = requests.post(
    url="http://localhost:8000/v1/chat/completions",
    json={
        "model": "Qwen/Qwen3-4B",
        "messages": [
            {"role": "system", "content": "Please keep the answer short and concise."},
            {"role": "user", "content": "Who are you? Reply in one sentence."},
        ],
    },
)
```

Experiment Summary

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.304	/	311.48	/
2	Prompt Engineering	1.127	+270.7%	377.41	+21.2%

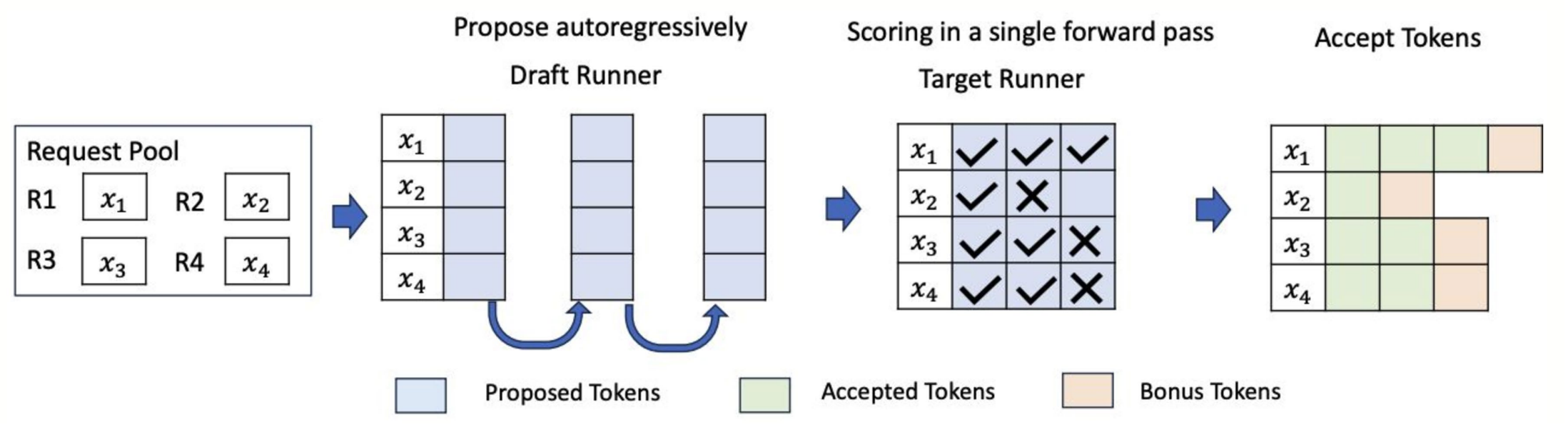
Model: Qwen/Qwen3-4B
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	1.111	/	1275.15	/
2	Prompt Engineering	2.947	+165.3%	1222.96	-4.09%

Model: Qwen/Qwen3-32B
GPU: NVIDIA B200 (180 GB VRAM)

Speculative Decoding

- ❌ **Predict** next token
- ⭕ **Verify** the token predicted by smaller model



[1] How Speculative Decoding Boosts vLLM Performance by up to 2.8x

Speculative Decoding in vLLM

- Just one line of change → good to go

```
wyhwong@wyhwong-desktop:~/projects/RedHat-vLLM-inferencing$ uv run --active \
    --with vllm \
    vllm serve Qwen/Qwen3-4B \
    --max_model_len 8192 \
    --override-generation-config '{"temperature": 0}'
```

↓

```
wyhwong@wyhwong-desktop:~/projects/RedHat-vLLM-inferencing$ uv run --active \
    --with vllm \
    vllm serve Qwen/Qwen3-4B \
    --max_model_len 8192 \
    --override-generation-config '{"temperature": 0}' \
    --speculative_config '{"method": "ngram", "model": "Qwen/Qwen3-0.6B", "num_speculative_tokens": 5}'
```

Experiment Summary

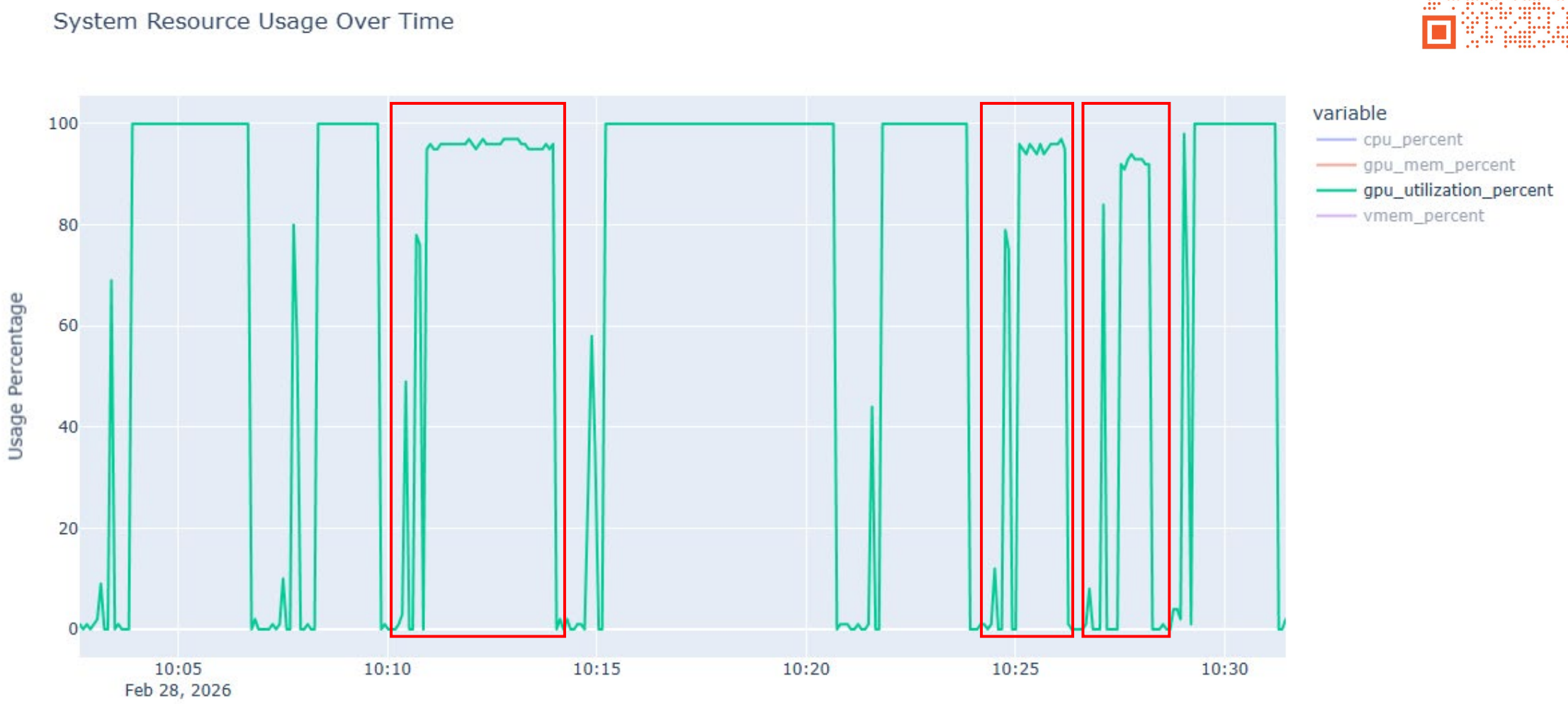
Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.304	/	311.48	/
3	Speculative Decoding	0.254	-16.4%	264.28	-15.2%

Target model: Qwen/Qwen3-4B
Draft model: Qwen/Qwen3-0.6B
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	1.111	/	1275.15	/
3	Speculative Decoding	0.952	-14.3%	1091.03	-14.4%

Target model: Qwen/Qwen3-32B
Draft model: Qwen/Qwen3-14B-AWQ
GPU: NVIDIA B200 (180 GB VRAM)

GPU Underutilization



Experiments with speculative decoding: GPU not fully utilized
Draft token acceptance rate too low → extra sampling work → lower throughput

Retry with Alternative Combination

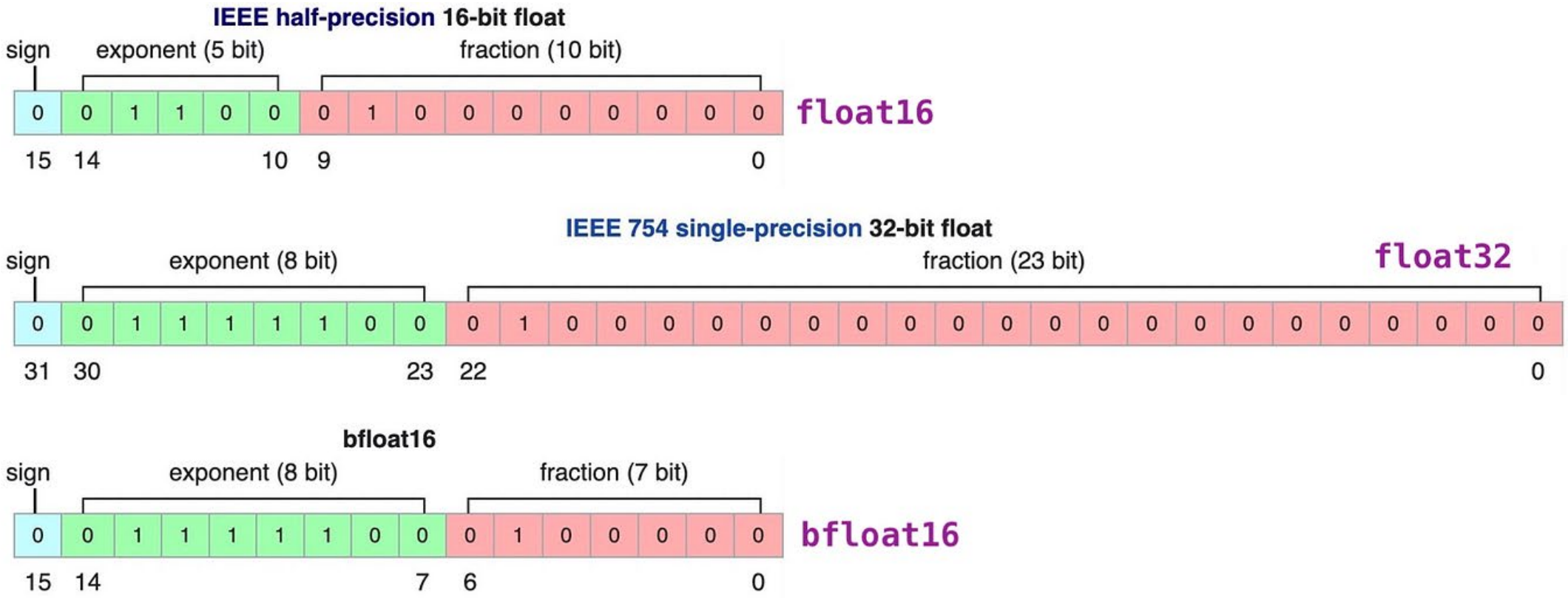
Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.016	/	31.53	/
3	Speculative Decoding	0.047	+193.8%	95.25	+202%

Target model: facebook/opt-6.7b
Draft model: facebook/opt-125m
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)

- The draft token acceptance rate greatly enhanced from ~30% to ~80%.
- Speculative decoding **may or may** not boost inference performance, **test** before using it.

Quantization

- Representing model weights with low-precision data types.



[1] LLM Series - Quantization Overview, 2023.

Experiment Summary

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.304	/	311.48	/
4	Quantization (vLLM)	0.151	-50.3%	179.21	-42.5%
5	Quantization (Pre-quantized)	0.558	+83.6%	646.41	+107.5%

Model: Qwen/Qwen3-4B (Pre-quantized: Qwen/Qwen3-4B-AWQ)
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	1.111	/	1275.15	/
4	Quantization (vLLM)	0.446	-59.9%	473.59	-62.9%
5	Quantization (Pre-quantized)	1.414	+27.3%	1645.24	**+29%**

Model: Qwen/Qwen3-32B (Pre-quantized: Qwen/Qwen3-32B-AWQ)
GPU: NVIDIA B200 (180 GB VRAM)

Local Experiment Summary

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.304	/	311.48	/
2	Prompt Engineering	1.127	+270.7%	377.41	+21.2%
3	Speculative Decoding	0.254	-16.4%	264.28	-15.2%
4	Quantization (vLLM)	0.151	**-50.3%**	179.21	**-42.5%**
5	Quantization (Pre-quantized)	0.558	+83.6%	646.41	+107.5%
6	1+2+3	0.981	+222.7%	321.18	+3.1%
7	2+3+5	1.347	+343.1%	543.46	+74.5%
8	2+5	2.093	**+588.5%**	861.71	**+176.7%**

Target model: Qwen/Qwen3-4B
Target model (Pre-quantized): Qwen/Qwen3-4B-AWQ
Draft model: Qwen/Qwen3-0.6B
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)

Cloud Experiment Summary

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	1.111	/	1275.15	/
2	Prompt Engineering	2.947	+165.3%	1222.96	-4.09%
3	Speculative Decoding	0.952	-14.3%	1091.03	-14.4%
4	Quantization (vLLM)	0.446	**-59.9%**	473.59	**-62.9%**
5	Quantization (Pre-quantized)	1.414	+27.3%	1645.24	**+29%**
6	1+2+3	2.001	+80.1%	829.51	-34.9%
7	2+3+5	2.757	+148.2%	1121.12	-12.1%
8	2+5	3.264	**+193.8%**	1656.87	+29.9%

Target model: Qwen/Qwen3-32B
Target model (Pre-quantized): Qwen/Qwen3-32B-AWQ
Draft model: Qwen/Qwen3-14B-AWQ
GPU: NVIDIA B200 (180 GB VRAM)

Q & A

References

- Explaining the Code of the vLLM Inference Engine, 2024. https://medium.com/@crclq2018/explaining-the-source-code-behind-the-vllm-fast-inference-engine-91429f54d1f7
- Advances in Deep Learning, 2025. https://ut.philkr.net/advances_in_deeplearning/
- vLLM: An Efficient Inference Engine for Large Language Models, 2025. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/EECS-2025-192.pdf
- Ollama vs. vLLM: A deep dive into performance benchmarking, 2025. https://developers.redhat.com/articles/2025/08/08/ollama-vs-vllm-deep-dive-performance-benchmarking#
- Performance vs Practicality: A Comparison of vLLM and Ollama, 2025. https://robert-mcdermott.medium.com/performance-vs-practicality-a-comparison-of-vllm-and-ollama-104acad250fd
- How Speculative Decoding Boosts vLLM Performance by up to 2.8x, 2024. https://blog.vllm.ai/2024/10/17/spec-decode.html
- Fast Inference from Transformers via Speculative Decoding, 2022. https://arxiv.org/abs/2211.17192
- vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention, 2023. https://blog.vllm.ai/2023/06/20/vllm.html
- Inside vLLM: Anatomy of a High-Throughput LLM Inference System, 2025. https://blog.vllm.ai/2025/09/05/anatomy-of-vllm.html

- Acknowledgement: Thanks Alex Au for sponsoring the testing on Runpod using NVIDIA B200 GPU.

Appendix: Ollama Concurrency Tests

Metric	Unit	Ollama		
Throughput	Request/second	0.116	0.144	0.069
	Token/second	166.57	216.00	102.99
Concurrency	/	4	8	12

Setup Summary:

- Model: qwen3:4b (Ollama)
- GPU: Nvidia RTX 4060 Ti (16GB VRAM)
- Data: 50 requests randomly sampled from Alpaca

Local Quantization Experiments

Setup	Settings	Throughput (Request/second)	Enhancement	Throughput (Token/second)	Enhancement
1	Vanilla	0.304	/	311.48	/
4	Quantization (vLLM, bitsandbytes)	0.151	-50.3%	179.21	-42.5%
4	Quantization (vLLM, FP8)	0.061	-79.9%	62.04	-80.0%
5	Quantization (Pre-quantized)	0.558	+83.6%	646.41	+107.5%

Model: Qwen/Qwen3-4B (Pre-quantized: Qwen/Qwen3-4B-AWQ)
GPU: Nvidia RTX 4060 Ti (16 GB VRAM)